# Migration_Python_Codebase_To_Go

# qBitTorrent Backend Migration: Python/FastAPI → Go/Gin

**Revision:** 1 **Last modified:** 2026-06-06T00:00:00Z

**Version:** 1.0.0
**Date:** 2026-04-22
**Author:** Migration Task Force
**Status:** Production-Ready Blueprint

---

## Table of Contents



---

# 1. Project Analysis & Pre-Migration Checklist

## 1.1 Source of Truth

- **OpenAPI Specification**: `http://localhost:7187/openapi.json`
  This file defines every endpoint, request/response schema, and error code.
  **Action**: Export it and keep it open for reference.
- **Existing Shell Scripts**: `start.sh`, `install-plugin.sh`, `stop.sh` remain **unchanged**.
- **Angular Frontend**: Located in `frontend/`. No code changes required.

## 1.2 Python Code Audit (Hypothetical – Based on OpenAPI)

The Python backend consists of: - `main.py` – FastAPI application entry point. - `app/api/search.py` – `/api/v1/search` endpoint with SSE streaming. - `app/api/hooks.py` – Hook registration and trigger logic. - `app/api/bridge.py` – Private tracker authentication and download. - `app/core/config.py` – Environment variable loading. - `app/models/*.py` – Pydantic models for requests/responses.

## 1.3 Dependencies to Replace

| Python Dependency | Go Replacement |
|---|---|
| `fastapi` | `gin-gonic/gin` |
| `uvicorn` | Built-in `net/http` server |
| `pydantic` | Struct tags + `validator` package |
| `httpx` / `requests` | `net/http` + custom client |
| `asyncio` | Goroutines + channels |
| `python-dotenv` | `godotenv` or `viper` |
| `sse-starlette` | Manual SSE with `http.Flusher` |

## 1.4 Plugin System Status

- **Nova3 search plugins** are Python scripts executed **by the qBittorrent client itself**, not by our backend.
- Our backend only calls qBittorrent's Web API (`/api/v2/search/start`, `/api/v2/search/results`, etc.).
- **Conclusion**: Plugins will work unchanged. The migration only affects the orchestrator.

---

# 2. Go Project Initialization & Structure

## 2.1 Initialize Module

```
mkdir qBitTorrent-go && cd qBitTorrent-go
go mod init github.com/milos85vasic/qBitTorrent-go
```

## 2.2 Directory Layout

```
.
├── cmd/
│   ├── qbittorrent-proxy/      # Main REST API server
│   │   └── main.go
│   └── webui-bridge/           # Private tracker bridge (separate
binary)
│       └── main.go
├── internal/
│   ├── api/                    # Gin handlers grouped by feature
│   │   ├── search.go
│   │   ├── hooks.go
│   │   ├── bridge.go
│   │   └── health.go
│   ├── client/                 # qBittorrent Web API client
│   │   ├── client.go
│   │   ├── auth.go
│   │   ├── torrents.go
│   │   └── search.go
│   ├── config/                 # Configuration structs and loader
│   │   └── config.go
│   ├── models/                 # Shared data structures
│   │   ├── torrent.go
│   │   ├── search.go
│   │   └── hook.go
│   ├── service/                # Business logic
│   │   ├── merge_search.go
│   │   ├── sse_broker.go
│   │   └── bridge_auth.go
│   └── middleware/             # Gin middleware
│       ├── cors.go
│       ├── logging.go
│       └── auth.go
├── pkg/                        # Public reusable packages (if
any)
├── scripts/                    # Build/deploy scripts
│   └── build.sh
├── frontend/                   # Angular frontend (unchanged)
├── .env.example
├── Dockerfile
├── docker-compose.yml
```

```
├── go.mod
└── go.sum
```

---

# 3. Data Models Translation (Pydantic → Go Structs)

## 3.1 Example: `Torrent` Model

### Python (Pydantic) – `app/models/torrent.py`

```python
from pydantic import BaseModel
from typing import Optional

class Torrent(BaseModel):
    hash: str
    name: str
    size: int
    progress: float
    state: Optional[str] = None
    category: Optional[str] = None
```

### Go – `internal/models/torrent.go`

```go
package models

type Torrent struct {
    Hash     string  `json:"hash"`
    Name     string  `json:"name"`
    Size     int64   `json:"size"`
    Progress float64 `json:"progress"`
    State    *string `json:"state,omitempty"`
    Category *string `json:"category,omitempty"`
}
```

## 3.2 Search Request/Response Models

**From OpenAPI:** - `POST /api/v1/search` expects: `json { "query": "ubuntu", "plugins": ["all"], "category": "all" }` - Response: Stream of SSE events with `Torrent` objects.

### Go – `internal/models/search.go`

```go
package models

type SearchRequest struct {
    Query    string   `json:"query" binding:"required"`
    Plugins  []string `json:"plugins" binding:"required"`
    Category string   `json:"category"`
```

```
    Limit    int      `json:"limit,omitempty"`
}

type SearchResult struct {
    Tracker  string   `json:"tracker"`
    Torrents []Torrent `json:"torrents"`
}
```

### 3.3 Hook Model

**Go – `internal/models/hook.go`**

```
package models

type Hook struct {
    ID       string            `json:"id"`
    URL      string            `json:"url" binding:"required,url"`
    Secret   string            `json:"secret,omitempty"`
    Events   []string          `json:"events" binding:"required"`
    Headers  map[string]string `json:"headers,omitempty"`
    Enabled  bool              `json:"enabled"`
}
```

---

# 4. Configuration Management

### 4.1 Configuration Struct

**File:** `internal/config/config.go`

```
package config

import (
    "log"
    "os"
    "strconv"

    "github.com/joho/godotenv"
)

type Config struct {
    QBittorrentURL      string
    QBittorrentUsername  string
    QBittorrentPassword  string
    ServerPort           string
    BridgePort           string
    LogLevel             string
    SSETimeout           int
    PluginTimeout        int
```

```
    PrivateTrackerConfig map[string]TrackerAuth
}

type TrackerAuth struct {
    Username string
    Password string
    LoginURL string
}

func Load() *Config {
    _ = godotenv.Load() // Ignore error if .env missing

    return &Config{
        QBittorrentURL:      getEnv("QBITTORRENT_URL", "http://
        localhost:8080"),
        QBittorrentUsername: getEnv("QBITTORRENT_USERNAME",
        "admin"),
        QBittorrentPassword: getEnv("QBITTORRENT_PASSWORD",
        "adminadmin"),
        ServerPort:          getEnv("SERVER_PORT", "7187"),
        BridgePort:          getEnv("BRIDGE_PORT", "7188"),
        LogLevel:            getEnv("LOG_LEVEL", "info"),
        SSETimeout:          getEnvAsInt("SSE_TIMEOUT", 30),
        PluginTimeout:       getEnvAsInt("PLUGIN_TIMEOUT", 10),
        // Load private tracker credentials from env like
        TRACKER_IPT_USERNAME, etc.
        PrivateTrackerConfig: loadTrackerAuth(),
    }
}

func getEnv(key, fallback string) string {
    if value, exists := os.LookupEnv(key); exists {
        return value
    }
    return fallback
}

func getEnvAsInt(key string, fallback int) int {
    if value, exists := os.LookupEnv(key); exists {
        if i, err := strconv.Atoi(value); err == nil {
            return i
        }
    }
    return fallback
}

func loadTrackerAuth() map[string]TrackerAuth {
    // Implementation to read env vars like TRACKER_IPT_USERNAME
        etc.
    // Returns map[trackerID]TrackerAuth
    return make(map[string]TrackerAuth)
}
```

### 4.2 Environment File Example

#### .env.example

```
QBITTORRENT_URL=http://localhost:8080
QBITTORRENT_USERNAME=admin
QBITTORRENT_PASSWORD=adminadmin
SERVER_PORT=7187
BRIDGE_PORT=7188
LOG_LEVEL=debug
SSE_TIMEOUT=30
PLUGIN_TIMEOUT=10
TRACKER_IPT_USERNAME=your_ipt_user
TRACKER_IPT_PASSWORD=your_ipt_pass
```

---

# 5. qBittorrent API Client Implementation

## 5.1 Client Structure

**File:** `internal/client/client.go`

```go
package client

import (
    "bytes"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "net/http/cookiejar"
    "net/url"
    "sync"
    "time"
)

type Client struct {
    BaseURL    *url.URL
    HTTPClient *http.Client
    mu         sync.RWMutex
    sid        string // qBittorrent session cookie value
}

func NewClient(baseURL, username, password string) (*Client,
        error) {
    u, err := url.Parse(baseURL)
    if err != nil {
        return nil, err
    }
```

```
    jar, _ := cookiejar.New(nil)
    c := &Client{
        BaseURL: u,
        HTTPClient: &http.Client{
            Jar:     jar,
            Timeout: 30 * time.Second,
        },
    }

    if err := c.Login(username, password); err != nil {
        return nil, err
    }
    return c, nil
}
```

## 5.2 Authentication

**File:** `internal/client/auth.go`

```
package client

import (
    "fmt"
    "net/url"
)

func (c *Client) Login(username, password string) error {
    loginURL := c.BaseURL.ResolveReference(&url.URL{Path: "/api/
        v2/auth/login"})
    data := url.Values{}
    data.Set("username", username)
    data.Set("password", password)

    resp, err := c.HTTPClient.PostForm(loginURL.String(), data)
    if err != nil {
        return err
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("login failed: %s", resp.Status)
    }

    // Extract SID cookie (qBittorrent sets "SID")
    for _, cookie := range c.HTTPClient.Jar.Cookies(loginURL) {
        if cookie.Name == "SID" {
            c.mu.Lock()
            c.sid = cookie.Value
            c.mu.Unlock()
            break
```

```
        }
    }
    return nil
}

func (c *Client) IsAuthenticated() bool {
    c.mu.RLock()
    defer c.mu.RUnlock()
    return c.sid != ""
}
```

## 5.3 Search Endpoints

**File:** `internal/client/search.go`

```
package client

import (
    "encoding/json"
    "fmt"
    "net/http"
    "net/url"
    "strconv"

    "github.com/milos85vasic/qBitTorrent-go/internal/models"
)

// StartSearch starts a new search job and returns its ID.
func (c *Client) StartSearch(query string, plugins []string,
        category string) (int, error) {
    searchURL := c.BaseURL.ResolveReference(&url.URL{Path: "/api/
        v2/search/start"})
    data := url.Values{}
    data.Set("pattern", query)
    data.Set("category", category)
    for _, p := range plugins {
        data.Add("plugins", p)
    }

    resp, err := c.HTTPClient.PostForm(searchURL.String(), data)
    if err != nil {
        return 0, err
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return 0, fmt.Errorf("search start failed: %s",
        resp.Status)
    }

    var result struct {
        ID int `json:"id"`
```

```
    }
    if err := json.NewDecoder(resp.Body).Decode(&result); err !=
        nil {
        return 0, err
    }
    return result.ID, nil
}

// GetSearchResults retrieves results for a given search ID.
func (c *Client) GetSearchResults(searchID int, limit, offset
        int) ([]models.Torrent, int, error) {
    resultsURL := c.BaseURL.ResolveReference(&url.URL{Path: "/api/
        v2/search/results"})
    q := resultsURL.Query()
    q.Set("id", strconv.Itoa(searchID))
    if limit > 0 {
        q.Set("limit", strconv.Itoa(limit))
    }
    if offset > 0 {
        q.Set("offset", strconv.Itoa(offset))
    }
    resultsURL.RawQuery = q.Encode()

    resp, err := c.HTTPClient.Get(resultsURL.String())
    if err != nil {
        return nil, 0, err
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return nil, 0, fmt.Errorf("failed to get results: %s",
        resp.Status)
    }

    var response struct {
        Results []models.Torrent `json:"results"`
        Total   int              `json:"total"`
        Status  string           `json:"status"`
    }
    if err := json.NewDecoder(resp.Body).Decode(&response); err !
        = nil {
        return nil, 0, err
    }
    return response.Results, response.Total, nil
}

// StopSearch stops a running search.
func (c *Client) StopSearch(searchID int) error {
    stopURL := c.BaseURL.ResolveReference(&url.URL{Path: "/api/v2/
        search/stop"})
    data := url.Values{}
    data.Set("id", strconv.Itoa(searchID))
```

```
    resp, err := c.HTTPClient.PostForm(stopURL.String(), data)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("failed to stop search: %s",
        resp.Status)
    }
    return nil
}

// ListPlugins returns all installed search plugins.
func (c *Client) ListPlugins() ([]map[string]interface{}, error) {
    pluginsURL := c.BaseURL.ResolveReference(&url.URL{Path: "/api/
        v2/search/plugins"})
    resp, err := c.HTTPClient.Get(pluginsURL.String())
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("failed to list plugins: %s",
        resp.Status)
    }
    var plugins []map[string]interface{}
    if err := json.NewDecoder(resp.Body).Decode(&plugins); err !=
        nil {
        return nil, err
    }
    return plugins, nil
}
```

## 5.4 Torrent Management Endpoints

**File:** `internal/client/torrents.go`

```
package client

import (
    "encoding/json"
    "fmt"
    "net/url"

    "github.com/milos85vasic/qBitTorrent-go/internal/models"
)

// GetTorrents returns a list of all torrents.
func (c *Client) GetTorrents() ([]models.Torrent, error) {
    infoURL := c.BaseURL.ResolveReference(&url.URL{Path: "/api/v2/
        torrents/info"})
    resp, err := c.HTTPClient.Get(infoURL.String())
```

```
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("failed to get torrents: %s",
        resp.Status)
    }
    var torrents []models.Torrent
    if err := json.NewDecoder(resp.Body).Decode(&torrents); err !
        = nil {
        return nil, err
    }
    return torrents, nil
}

// AddTorrent adds a new torrent from URL or file.
func (c *Client) AddTorrent(torrentURL, savepath, category
        string) error {
    addURL := c.BaseURL.ResolveReference(&url.URL{Path: "/api/v2/
        torrents/add"})
    data := url.Values{}
    data.Set("urls", torrentURL)
    if savepath != "" {
        data.Set("savepath", savepath)
    }
    if category != "" {
        data.Set("category", category)
    }

    resp, err := c.HTTPClient.PostForm(addURL.String(), data)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("add torrent failed: %s", resp.Status)
    }
    return nil
}
```

---

# 6. Core Business Logic Migration

## 6.1 Merge Search Service

**Goal:** Replace Python `asyncio.gather` with goroutines to query multiple plugins concurrently and stream results via SSE.

**File:** `internal/service/merge_search.go`

```go
package service

import (
    "context"
    "fmt"
    "sync"
    "time"

    "github.com/milos85vasic/qBitTorrent-go/internal/client"
    "github.com/milos85vasic/qBitTorrent-go/internal/models"
    "github.com/rs/zerolog/log"
)

type MergeSearchService struct {
    qbitClient *client.Client
}

func NewMergeSearchService(qc *client.Client) *MergeSearchService
        {
    return &MergeSearchService{qbitClient: qc}
}

// SearchResultBatch represents a batch of torrents from a
        tracker.
type SearchResultBatch struct {
    Tracker  string
    Torrents []models.Torrent
    Error    error
}

// Search performs concurrent searches across all specified
        plugins.
func (s *MergeSearchService) Search(ctx context.Context, req
        models.SearchRequest) (<-chan SearchResultBatch, error) {
    if len(req.Plugins) == 0 {
        return nil, fmt.Errorf("no plugins specified")
    }

    out := make(chan SearchResultBatch, len(req.Plugins))
    var wg sync.WaitGroup

    for _, plugin := range req.Plugins {
        wg.Add(1)
        go func(p string) {
            defer wg.Done()
            s.searchPlugin(ctx, req, p, out)
        }(plugin)
    }

    go func() {
        wg.Wait()
        close(out)
```

```
    }()

    return out, nil
}

func (s *MergeSearchService) searchPlugin(ctx
        context.Context, req models.SearchRequest, plugin string,
        out chan<- SearchResultBatch) {
    // Create a timeout context for this plugin search
    pluginCtx, cancel := context.WithTimeout(ctx,
        time.Duration(s.qbitClient.PluginTimeout)*time.Second)
    defer cancel()

    // Start search on qBittorrent
    searchID, err := s.qbitClient.StartSearch(req.Query,
        []string{plugin}, req.Category)
    if err != nil {
        out <- SearchResultBatch{Tracker: plugin, Error: err}
        return
    }
    defer func() {
        _ = s.qbitClient.StopSearch(searchID)
    }()

    // Poll for results until completion or context done
    ticker := time.NewTicker(500 * time.Millisecond)
    defer ticker.Stop()

    var allTorrents []models.Torrent
    for {
        select {
        case <-pluginCtx.Done():
            out <- SearchResultBatch{Tracker: plugin, Error:
        pluginCtx.Err()}
            return
        case <-ticker.C:
            torrents, total, err :=
        s.qbitClient.GetSearchResults(searchID, 100,
        len(allTorrents))
            if err != nil {
                log.Warn().Err(err).Str("plugin",
        plugin).Msg("failed to fetch results")
                continue
            }
            allTorrents = append(allTorrents, torrents...)
            // Send batch immediately for SSE streaming
            if len(torrents) > 0 {
                out <- SearchResultBatch{Tracker: plugin,
        Torrents: torrents}
            }
            // Check if search is complete (status "Stopped" or
        all results fetched)
            if len(allTorrents) >= total || total == 0 {
```

```
                return
            }
        }
    }
}
```

## 6.2 Private Tracker Bridge (`webui-bridge.py` → `cmd/webui-bridge/main.go`)

**Purpose:** Handle authentication to private trackers and download `.torrent` files that require login.

**File:** `cmd/webui-bridge/main.go`

```
package main

import (
    "bytes"
    "fmt"
    "io"
    "net/http"
    "net/http/cookiejar"
    "net/url"
    "os"
    "strings"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/milos85vasic/qBitTorrent-go/internal/config"
)

var trackerAuth = map[string]config.TrackerAuth{
    "iptorrents": {
        LoginURL: "https://iptorrents.com/torrents/",
        // Username/Password loaded from env
    },
    // Add other trackers...
}

func main() {
    cfg := config.Load()
    r := gin.Default()

    // Health endpoint
    r.GET("/bridge/health", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{"status": "ok"})
    })

    // Download endpoint
    r.GET("/bridge/download", downloadHandler(cfg))
```

```
    r.Run(":" + cfg.BridgePort)
}

func downloadHandler(cfg *config.Config) gin.HandlerFunc {
    return func(c *gin.Context) {
        tracker := c.Query("tracker")
        torrentURL := c.Query("url")
        if tracker == "" || torrentURL == "" {
            c.JSON(http.StatusBadRequest, gin.H{"error": "tracker
        and url required"})
            return
        }

        // Load tracker-specific auth
        auth, ok := cfg.PrivateTrackerConfig[tracker]
        if !ok {
            c.JSON(http.StatusNotFound, gin.H{"error": "tracker
        not configured"})
            return
        }

        // Create HTTP client with cookie jar
        jar, _ := cookiejar.New(nil)
        client := &http.Client{
            Jar:     jar,
            Timeout: 30 * time.Second,
            CheckRedirect: func(req *http.Request, via
        []*http.Request) error {
                // Preserve cookies across redirects
                return nil
            },
        }

        // Perform login
        loginData := url.Values{}
        loginData.Set("username", auth.Username)
        loginData.Set("password", auth.Password)
        loginReq, _ := http.NewRequest("POST", auth.LoginURL,
        strings.NewReader(loginData.Encode()))
        loginReq.Header.Set("Content-Type", "application/x-www-
        form-urlencoded")
        loginReq.Header.Set("User-Agent", "Mozilla/5.0")

        loginResp, err := client.Do(loginReq)
        if err != nil {
            c.JSON(http.StatusInternalServerError, gin.H{"error":
        "login failed: " + err.Error()})
            return
        }
        defer loginResp.Body.Close()
        if loginResp.StatusCode != http.StatusOK {
            body, _ := io.ReadAll(loginResp.Body)
```

```
            c.JSON(http.StatusUnauthorized, gin.H{"error":
        fmt.Sprintf("login rejected: %s", body)})
            return
        }

        // Now download the torrent file
        downloadReq, _ := http.NewRequest("GET", torrentURL, nil)
        downloadReq.Header.Set("User-Agent", "Mozilla/5.0")
        downloadResp, err := client.Do(downloadReq)
        if err != nil {
            c.JSON(http.StatusInternalServerError, gin.H{"error":
        "download failed: " + err.Error()})
            return
        }
        defer downloadResp.Body.Close()

        if downloadResp.StatusCode != http.StatusOK {
            c.JSON(downloadResp.StatusCode, gin.H{"error":
        "torrent not found"})
            return
        }

        // Stream the torrent file to the client
        c.Header("Content-Disposition", "attachment;
        filename=torrent.torrent")
        c.Header("Content-Type", "application/x-bittorrent")
        _, err = io.Copy(c.Writer, downloadResp.Body)
        if err != nil {
            log.Error().Err(err).Msg("failed to stream torrent
        file")
        }
    }
}
```

## 6.3 Server-Sent Events (SSE) Streaming

**File:** `internal/service/sse_broker.go`

```
package service

import (
    "fmt"
    "io"
    "sync"

    "github.com/gin-gonic/gin"
    "github.com/rs/zerolog/log"
)

type SSEBroker struct {
    clients map[chan string]bool
    mu      sync.RWMutex
```

```
}

func NewSSEBroker() *SSEBroker {
    return &SSEBroker{
        clients: make(map[chan string]bool),
    }
}

// Subscribe adds a client channel and returns a function to
        unsubscribe.
func (b *SSEBroker) Subscribe() (chan string, func()) {
    b.mu.Lock()
    defer b.mu.Unlock()
    ch := make(chan string, 10)
    b.clients[ch] = true
    return ch, func() {
        b.mu.Lock()
        defer b.mu.Unlock()
        delete(b.clients, ch)
        close(ch)
    }
}

// Publish sends a message to all subscribed clients.
func (b *SSEBroker) Publish(event string, data string) {
    b.mu.RLock()
    defer b.mu.RUnlock()
    msg := fmt.Sprintf("event: %s\ndata: %s\n\n", event, data)
    for ch := range b.clients {
        select {
        case ch <- msg:
        default:
            // Client is slow, drop message
            log.Warn().Msg("dropping SSE message for slow client")
        }
    }
}

// GinHandler returns a Gin handler that streams SSE.
func (b *SSEBroker) GinHandler() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Header("Content-Type", "text/event-stream")
        c.Header("Cache-Control", "no-cache")
        c.Header("Connection", "keep-alive")
        c.Header("Access-Control-Allow-Origin", "*")

        ch, unsubscribe := b.Subscribe()
        defer unsubscribe()

        clientGone := c.Request.Context().Done()
        for {
            select {
```

```
            case <-clientGone:
                log.Info().Msg("SSE client disconnected")
                return
            case msg := <-ch:
                _, err := io.WriteString(c.Writer, msg)
                if err != nil {
                    return
                }
                c.Writer.Flush()
            }
        }
    }
}
```

---

# 7. HTTP API Endpoints with Gin

## 7.1 Main Application Setup

**File:** `cmd/qbittorrent-proxy/main.go`

```
package main

import (
    "context"
    "encoding/json"
    "net/http"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/milos85vasic/qBitTorrent-go/internal/api"
    "github.com/milos85vasic/qBitTorrent-go/internal/client"
    "github.com/milos85vasic/qBitTorrent-go/internal/config"
    "github.com/milos85vasic/qBitTorrent-go/internal/middleware"
    "github.com/milos85vasic/qBitTorrent-go/internal/models"
    "github.com/milos85vasic/qBitTorrent-go/internal/service"
    "github.com/rs/zerolog/log"
)

func main() {
    cfg := config.Load()

    // Initialize qBittorrent client
    qbitClient, err := client.NewClient(cfg.QBittorrentURL,
        cfg.QBittorrentUsername, cfg.QBittorrentPassword)
    if err != nil {
        log.Fatal().Err(err).Msg("failed to connect to
        qBittorrent")
    }
```

```
// Initialize services
searchService := service.NewMergeSearchService(qbitClient)
sseBroker := service.NewSSEBroker()

// Gin router
r := gin.Default()
r.Use(middleware.CORS())
r.Use(middleware.Logger())

// Health check
r.GET("/health", func(c *gin.Context) {
    c.JSON(http.StatusOK, gin.H{"status": "ok"})
})

// API v1 group
v1 := r.Group("/api/v1")
{
    // Search endpoint with SSE streaming
    v1.POST("/search", func(c *gin.Context) {
        var req models.SearchRequest
        if err := c.ShouldBindJSON(&req); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error":
    err.Error()})
            return
        }

        // Set up SSE
        c.Header("Content-Type", "text/event-stream")
        c.Header("Cache-Control", "no-cache")
        c.Header("Connection", "keep-alive")
        c.Writer.Flush()

        ctx, cancel :=
    context.WithTimeout(c.Request.Context(),
    time.Duration(cfg.SSETimeout)*time.Second)
        defer cancel()

        results, err := searchService.Search(ctx, req)
        if err != nil {
            sseBroker.Publish("error", err.Error())
            return
        }

        for batch := range results {
            if batch.Error != nil {
                sseBroker.Publish("error",
    batch.Error.Error())
                continue
            }
            data, _ := json.Marshal(batch)
            sseBroker.Publish("results", string(data))
            c.Writer.Flush()
```

```
            }
            sseBroker.Publish("done", "{}")
        })

        // Hook management endpoints (stub)
        hooks := v1.Group("/hooks")
        {
            hooks.GET("", api.ListHooks)
            hooks.POST("", api.CreateHook)
            hooks.DELETE("/:id", api.DeleteHook)
        }

        // Private tracker bridge proxy (if needed)
        v1.GET("/bridge/download", api.ProxyBridgeDownload(cfg))
    }

    // Start server
    log.Info().Str("port", cfg.ServerPort).Msg("starting server")
    if err := r.Run(":" + cfg.ServerPort); err != nil {
        log.Fatal().Err(err).Msg("server failed")
    }
}
```

## 7.2 Hook Handlers Example

**File:** `internal/api/hooks.go`

```
package api

import (
    "net/http"

    "github.com/gin-gonic/gin"
    "github.com/milos85vasic/qBitTorrent-go/internal/models"
)

var hooksStore = make(map[string]models.Hook) // In production,
        use a database

func ListHooks(c *gin.Context) {
    hooks := make([]models.Hook, 0, len(hooksStore))
    for _, h := range hooksStore {
        hooks = append(hooks, h)
    }
    c.JSON(http.StatusOK, hooks)
}

func CreateHook(c *gin.Context) {
    var hook models.Hook
    if err := c.ShouldBindJSON(&hook); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
```

```
        return
    }
    hook.ID = generateID()
    hook.Enabled = true
    hooksStore[hook.ID] = hook
    c.JSON(http.StatusCreated, hook)
}

func DeleteHook(c *gin.Context) {
    id := c.Param("id")
    if _, ok := hooksStore[id]; !ok {
        c.JSON(http.StatusNotFound, gin.H{"error": "hook not
        found"})
        return
    }
    delete(hooksStore, id)
    c.Status(http.StatusNoContent)
}
```

---

# 8. Middleware & Cross-Cutting Concerns

## 8.1 CORS Middleware

**File:** `internal/middleware/cors.go`

```
package middleware

import "github.com/gin-gonic/gin"

func CORS() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Writer.Header().Set("Access-Control-Allow-Origin", "*")
        c.Writer.Header().Set("Access-Control-Allow-Credentials",
        "true")
        c.Writer.Header().Set("Access-Control-Allow-Headers",
        "Content-Type, Content-Length, Accept-Encoding, X-CSRF-
        Token, Authorization, accept, origin, Cache-Control, X-
        Requested-With")
        c.Writer.Header().Set("Access-Control-Allow-Methods",
        "POST, OPTIONS, GET, PUT, DELETE")

        if c.Request.Method == "OPTIONS" {
            c.AbortWithStatus(204)
            return
        }
        c.Next()
```

```
    }
}
```

### 8.2 Structured Logging with Zerolog

**File:** `internal/middleware/logging.go`

```go
package middleware

import (
    "time"

    "github.com/gin-gonic/gin"
    "github.com/rs/zerolog/log"
)

func Logger() gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()
        path := c.Request.URL.Path
        raw := c.Request.URL.RawQuery

        c.Next()

        latency := time.Since(start)
        if raw != "" {
            path = path + "?" + raw
        }

        log.Info().
            Str("method", c.Request.Method).
            Str("path", path).
            Int("status", c.Writer.Status()).
            Dur("latency", latency).
            Str("client_ip", c.ClientIP()).
            Msg("request")
    }
}
```

---

# 9. Testing Strategy & Implementation

### 9.1 Unit Tests for qBittorrent Client

**File:** `internal/client/client_test.go`

```go
package client

import (
```

```
    "net/http"
    "net/http/httptest"
    "testing"
)

func TestLogin(t *testing.T) {
    server := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
        if r.URL.Path == "/api/v2/auth/login" && r.Method ==
        "POST" {
            http.SetCookie(w, &http.Cookie{Name: "SID", Value:
        "test-sid"})
            w.WriteHeader(http.StatusOK)
            return
        }
        w.WriteHeader(http.StatusNotFound)
    }))
    defer server.Close()

    client, err := NewClient(server.URL, "user", "pass")
    if err != nil {
        t.Fatalf("expected no error, got %v", err)
    }
    if !client.IsAuthenticated() {
        t.Error("expected client to be authenticated")
    }
}
```

## 9.2 Integration Test for Search Flow

**File:** `internal/api/search_test.go`

```
package api

import (
    "bytes"
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "testing"

    "github.com/gin-gonic/gin"
    "github.com/milos85vasic/qBitTorrent-go/internal/models"
)

func TestSearchEndpoint(t *testing.T) {
    gin.SetMode(gin.TestMode)
    r := gin.Default()
    // ... setup dependencies and routes

    body, _ := json.Marshal(models.SearchRequest{
        Query:   "linux",
```

```
        Plugins: []string{"all"},
    })
    req, _ := http.NewRequest("POST", "/api/v1/search",
        bytes.NewReader(body))
    req.Header.Set("Content-Type", "application/json")
    w := httptest.NewRecorder()
    r.ServeHTTP(w, req)

    if w.Code != http.StatusOK {
        t.Errorf("expected 200, got %d", w.Code)
    }
    // Additional SSE parsing tests...
}
```

### 9.3 Run Tests

```
go test ./... -v
go test -race ./...
```

---

# 10. Docker & Deployment Configuration

### 10.1 Multi-Stage Dockerfile

**File:** `Dockerfile`

```
# Build stage
FROM golang:1.23-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /qbittorrent-proxy ./cmd/
        qbittorrent-proxy
RUN CGO_ENABLED=0 GOOS=linux go build -o /webui-bridge ./cmd/
        webui-bridge

# Final stage
FROM alpine:3.19
RUN apk --no-cache add ca-certificates tzdata
WORKDIR /app
COPY --from=builder /qbittorrent-proxy .
COPY --from=builder /webui-bridge .
COPY .env.example .env
EXPOSE 7187 7188
CMD ["/app/qbittorrent-proxy"]
```

## 10.2 Docker Compose

**File:** `docker-compose.yml`

```yaml
version: '3.8'
services:
  qbittorrent:
    image: linuxserver/qbittorrent:latest
    container_name: qbittorrent
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/London
      - WEBUI_PORT=8080
    volumes:
      - ./config/qbittorrent:/config
      - ./downloads:/downloads
      - ./plugins:/config/qBittorrent/nova3/engines
    ports:
      - "8080:8080"
      - "6881:6881"
      - "6881:6881/udp"
    restart: unless-stopped

  backend:
    build: .
    container_name: qbittorrent-proxy
    ports:
      - "7187:7187"
      - "7188:7188"
    environment:
      - QBITTORRENT_URL=http://qbittorrent:8080
      - QBITTORRENT_USERNAME=admin
      - QBITTORRENT_PASSWORD=adminadmin
    depends_on:
      - qbittorrent
    volumes:
      - ./plugins:/plugins:ro
    restart: unless-stopped

  frontend:
    build: ./frontend
    container_name: qbittorrent-frontend
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: unless-stopped
```

### 10.3 Update `start.sh`

The existing `start.sh` script should be updated to build the Go binaries and Docker images. Example snippet:

```
#!/bin/bash
echo "Building Go backend..."
go build -o bin/qbittorrent-proxy ./cmd/qbittorrent-proxy
go build -o bin/webui-bridge ./cmd/webui-bridge

echo "Building Angular frontend..."
cd frontend && npm install && npm run build --prod && cd ..

echo "Starting services with Docker Compose..."
docker-compose up -d --build
```

---

## 11. Plugin System Compatibility Assurance

### 11.1 How Plugins Are Called

The Go backend **does not execute** the Python Nova3 plugins. It delegates all search tasks to the qBittorrent client via its Web API. The qBittorrent client has its own Python interpreter and runs the plugins exactly as before.

### 11.2 Verification Steps

1. **List plugins** via Go backend: `GET /api/v2/search/plugins` (proxied to qBittorrent).
2. **Start a search** with `POST /api/v1/search` – this internally calls qBittorrent's `/api/v2/search/start`.
3. **Observe results** – the SSE stream contains torrents parsed by the plugins.

### 11.3 Plugin Installation

The `install-plugin.sh` script remains unchanged. It copies plugin files into the volume mounted to qBittorrent's `nova3/engines` directory. The Go backend does not interfere.

### 11.4 Edge Cases

- **Private trackers requiring login**: The `webui-bridge` service handles authentication and provides a download endpoint. The Go backend proxies download requests to this bridge.

- **Plugin timeouts**: The Go service respects the `PLUGIN_TIMEOUT` environment variable and cancels the context if a plugin hangs.

---

# 12. Post-Migration Validation Checklist

| # | Item | Verification Method |
|---|---|---|
| 1 | All `/api/v1/*` endpoints respond with expected status codes | Run `curl` against each endpoint |
| 2 | SSE search stream delivers results from multiple trackers | Use browser EventSource or `curl -N` |
| 3 | Hook registration and triggering works | Register a test webhook and verify payload |
| 4 | Private tracker downloads succeed | Test with a configured tracker (e.g., IPT) |
| 5 | Angular frontend loads and interacts correctly | Manual UI testing |
| 6 | Existing Nova3 plugins are listed and used | Check search results contain expected tracker names |
| 7 | Logs are structured and contain correlation IDs | Inspect Docker logs |
| 8 | No memory leaks or goroutine | Run with `pprof` and monitor |

| # | Item | Verification Method |
|---|---|---|
| | leaks under load | |
| 9 | Docker images build successfully | `docker-compose build` |
| 10 | Environment variables are correctly parsed | Add debug endpoint to dump config |

## 13. Appendix: File-by-File Migration Map

| Python File | Go Equivalent | Notes |
|---|---|---|
| `main.py` | `cmd/qbittorrent-proxy/main.go` | Gin router setup, service wiring |
| `app/api/search.py` | `internal/api/search.go` (and `service/merge_search.go`) | SSE streaming, concurrent plugin calls |
| `app/api/hooks.py` | `internal/api/hooks.go` | CRUD for webhooks |
| `app/api/bridge.py` | `cmd/webui-bridge/main.go` | Private tracker auth & download |
| `app/models/*.py` | `internal/models/*.go` | Structs with JSON tags |
| `app/core/config.py` | `internal/config/config.go` | Environment loading with godotenv |
| `webui-bridge.py` | `cmd/webui-bridge/main.go` | Standalone microservice |
| `requirements.txt` | `go.mod` | Dependency management |
| `openapi.json` | N/A (source of truth) | Used for API contract validation |
| `start.sh` | `scripts/build.sh` + updated `start.sh` | Build and orchestrate |

**End of Migration Guide**

This document provides a complete, actionable plan. Follow each section sequentially, and refer to the file map and code examples to ensure every line of Python logic is translated into Go. The result will be a performant, maintainable backend that works seamlessly with your existing Angular frontend and qBittorrent plugins.